

Operatori in C++

Operatori in C++

Gli operatori aritmetici, di confronto e logici in C++.

Cosa sono gli operatori?

Un programma senza operatori non può fare quasi nulla. Gli **operatori** sono i simboli che usano il programma per fare calcoli, confrontare valori e prendere decisioni.

Sono come i simboli matematici che già conosci (`+` , `-` , `*` , `/`), con qualche aggiunta specifica per la programmazione.

Operatori aritmetici

Questi operatori fanno calcoli matematici:

Operatore	Operazione	Esempio	Risultato
<code>+</code>	Addizione	<code>5 + 3</code>	<code>8</code>
<code>-</code>	Sottrazione	<code>5 - 3</code>	<code>2</code>
<code>*</code>	Moltiplicazione	<code>5 * 3</code>	<code>15</code>
<code>/</code>	Divisione	<code>10 / 3</code>	<code>3</code> (intera!)
<code>%</code>	Modulo (resto)	<code>10 % 3</code>	<code>1</code>

```
int a = 10, b = 3;

cout << a + b << endl; // 13
cout << a - b << endl; // 7
cout << a * b << endl; // 30
cout << a / b << endl; // 3 - attenzione, non 3.33!
cout << a % b << endl; // 1 - il resto della divisione
```

La divisione intera

Quando si dividono due numeri interi, il risultato è sempre un intero. La parte decimale viene scartata, non arrotondata:

```
int x = 7, y = 2;
cout << x / y << endl;           // 3 (non 3.5!)

// Per ottenere 3.5, converti uno dei due in double prima di dividere
double risultato = (double)x / y;
cout << risultato << endl;      // 3.5
```

Il modulo %

Il modulo restituisce il **resto** della divisione intera. È utile per capire se un numero è pari o dispari:

```
cout << 10 % 3 << endl; // 1 (10 = 3×3 + 1)
cout << 9 % 3 << endl;  // 0 (9 è divisibile per 3)
cout << 7 % 2 << endl;  // 1 (7 è dispari)
cout << 8 % 2 << endl;  // 0 (8 è pari)
```

Operatori di assegnazione

Oltre al semplice `=`, esistono operatori che modificano una variabile in un colpo solo:

Operatore	Equivalente	Esempio con <code>x = 10</code>
<code>=</code>	assegnazione semplice	<code>x = 5</code> → x vale 5
<code>+=</code>	<code>x = x + y</code>	<code>x += 3</code> → x vale 13
<code>-=</code>	<code>x = x - y</code>	<code>x -= 3</code> → x vale 7
<code>*=</code>	<code>x = x * y</code>	<code>x *= 2</code> → x vale 20
<code>/=</code>	<code>x = x / y</code>	<code>x /= 5</code> → x vale 2
<code>%=</code>	<code>x = x % y</code>	<code>x %= 3</code> → x vale 1

```
int x = 10;
x += 5;    // x è ora 15
x -= 3;    // x è ora 12
x *= 2;    // x è ora 24
x /= 4;    // x è ora 6
x %= 4;    // x è ora 2
```

Operatori di incremento e decremento

Aggiungere o sottrarre 1 è così comune in C++ che esistono operatori appositi:

```
int x = 5;
x++;    // x diventa 6 (aggiunge 1)
x--;    // x diventa 5 (sottrae 1)
```

Esistono anche nella forma con `++` prima della variabile:

```
int x = 5;
int b = x++;    // prima assegna x a b (b = 5), poi incrementa x (x = 6)
int c = ++x;    // prima incrementa x (x = 7), poi assegna a c (c = 7)
```

Per i principianti è sufficiente usare `x++` e `x--` da soli, senza usarli dentro espressioni complesse.

Operatori di confronto

Confrontano due valori e restituiscono `true` (vero) o `false` (falso):

Operatore	Significato	Esempio	Risultato
<code>==</code>	uguale a	<code>5 == 5</code>	<code>true</code>
<code>!=</code>	diverso da	<code>5 != 3</code>	<code>true</code>
<code><</code>	minore di	<code>3 < 5</code>	<code>true</code>
<code>></code>	maggiore di	<code>5 > 3</code>	<code>true</code>
<code><=</code>	minore o uguale	<code>5 <= 5</code>	<code>true</code>
<code>>=</code>	maggiore o uguale	<code>3 >= 5</code>	<code>false</code>

```
int a = 5, b = 3;
```

```
cout << (a == b) << endl; // 0 (false - 5 non è uguale a 3)
cout << (a != b) << endl; // 1 (true - 5 è diverso da 3)
cout << (a > b) << endl; // 1 (true - 5 è maggiore di 3)
cout << (a < b) << endl; // 0 (false - 5 non è minore di 3)
```

Errore comune: confondere `==` (confronto) con `=` (assegnazione). Scrivere `if (x = 5)` invece di `if (x == 5)` cambia il valore di `x` invece di confrontarlo!

Operatori logici

Combinano più condizioni insieme:

Operatore	Significato	Esempio	Risultato
<code>&&</code>	E (AND)	<code>true && false</code>	<code>false</code>
<code> </code>	O (OR)	<code>true false</code>	<code>true</code>
<code>!</code>	NON (NOT)	<code>!true</code>	<code>false</code>

AND (&&): entrambe le condizioni devono essere vere

```
int eta = 20;
bool haPatente = true;

if (eta >= 18 && haPatente) {
    cout << "Puoi guidare." << endl;
}
// Entrambe le condizioni sono vere → stampa il messaggio
```

OR (||): basta che almeno una condizione sia vera

```
int ora = 14;

if (ora < 9 || ora > 18) {
    cout << "Il negozio è chiuso." << endl;
} else {
    cout << "Il negozio è aperto." << endl;
}
// ora = 14, nessuna delle due condizioni è vera → "aperto"
```

NOT (!): inverte il valore booleano

```
bool piove = false;

if (!piove) {
    cout << "Possiamo uscire!" << endl;
}
// !false è true → stampa il messaggio
```

La priorità degli operatori

Come in matematica, alcuni operatori hanno la precedenza su altri. Da quella più alta a quella più bassa:

1. `!`, `++`, `--`
2. `*`, `/`, `%`
3. `+`, `-`
4. `<`, `>`, `<=`, `>=`
5. `==`, `!=`
6. `&&`
7. `||`
8. `=`, `+=`, `-=`, ecc.

Quando hai dubbi, usa le parentesi per rendere esplicito l'ordine:

```
int risultato = 2 + 3 * 4;    // 14 - la moltiplicazione va prima
int risultato2 = (2 + 3) * 4; // 20 - le parentesi cambiano tutto
```

Esempio pratico

```
#include <iostream>
using namespace std;

int main() {
    int a = 15, b = 4;

    // Operatori aritmetici
    cout << "Somma: " << a + b << endl;      // 19
    cout << "Differenza: " << a - b << endl;  // 11
    cout << "Prodotto: " << a * b << endl;    // 60
    cout << "Quoziente: " << a / b << endl;   // 3 (divisione intera)
    cout << "Resto: " << a % b << endl;      // 3

    cout << endl;

    // Operatori di confronto
    cout << "a > b: " << (a > b) << endl;    // 1 (true)
    cout << "a == b: " << (a == b) << endl; // 0 (false)

    // Operatori logici
    bool cond = (a > 10) && (b < 10);
    cout << "a>10 E b<10: " << cond << endl; // 1 (true)

    return 0;
}
```